SIMATS ENGINEERING

ASSESSMENT TOOL 1

NAME: Lochana Sri R.S.

REG.NO: 192521348

COURSE NAME: Computer Architecture COURSE

CODE: CSA1202

QUESTIONS:5 Total Marks:25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I Lochana Sri R.S./ 192521348 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Lochana Sri R.S.

Signature of the student

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system

Analysis

A smart city surveillance system continuously receives video streams from multiple CCTV cameras installed at different locations. These video frames are transferred from the input devices (cameras) to the main memory (RAM), where they are temporarily stored before being processed by the CPU. The CPU executes image processing and threat detection algorithms to identify suspicious activities.

During peak hours, hundreds of cameras simultaneously transmit high-resolution video data. This creates heavy traffic between the I/O devices, memory, and CPU. As a result, the CPU spends more time waiting for data from memory instead of executing instructions, reducing the overall performance of the surveillance system.

Interaction Between CPU, Memory and I/O

CPU

- Fetches instructions from memory.
- Executes image processing and threat detection algorithms.
- Sends processed results to monitoring systems.

Memory (Cache and RAM)

- Cache stores frequently accessed instructions and data for faster retrieval.
- RAM temporarily stores video frames before processing.
- Cache misses increase memory access time and delay execution.

I/O Devices

- Cameras continuously send video frames.
- Display units show processed video and alerts.
- Large data transfers increase I/O traffic.

When the CPU, memory, and I/O devices compete for the same communication path, data transfer becomes slower, causing delays in threat detection.

Bus Structures
Single-Bus Structure

- One common bus connects the CPU, memory, and I/O devices.
- Only one device can transfer data at a time.
- High bus contention occurs during peak traffic.
- Increased waiting time results in lower system performance.

Multi-Bus Structure

- Separate buses are provided for CPU, memory, and I/O communication.
- Multiple transfers occur simultaneously.
- Reduces bus contention.
- Improves throughput and decreases response time.

Hierarchical Bus Structure

- Uses local buses and a high-speed system bus.
- Frequently communicating components use local buses.
- Reduces overall bus traffic.
- Suitable for large surveillance systems with multiple processors.

Instruction Execution Cycle

The CPU performs every task using the Fetch–Decode–Execute cycle.

Fetch

The Control Unit fetches instructions and video frame addresses from memory.

Decode

The Control Unit interprets the instruction and determines the required operation.

Execute

The ALU processes image data and detects threats.

Store

The processed results are stored back in memory and displayed on monitoring screens.

Improving the execution cycle using pipelining allows multiple instructions to be executed simultaneously, thereby increasing system performance.

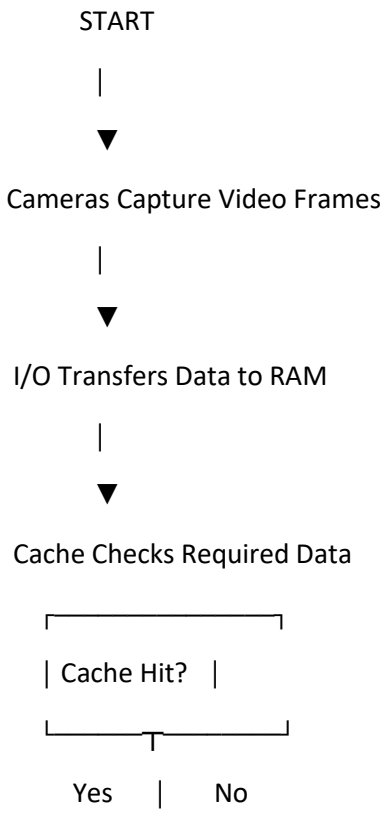
Possible Bottlenecks

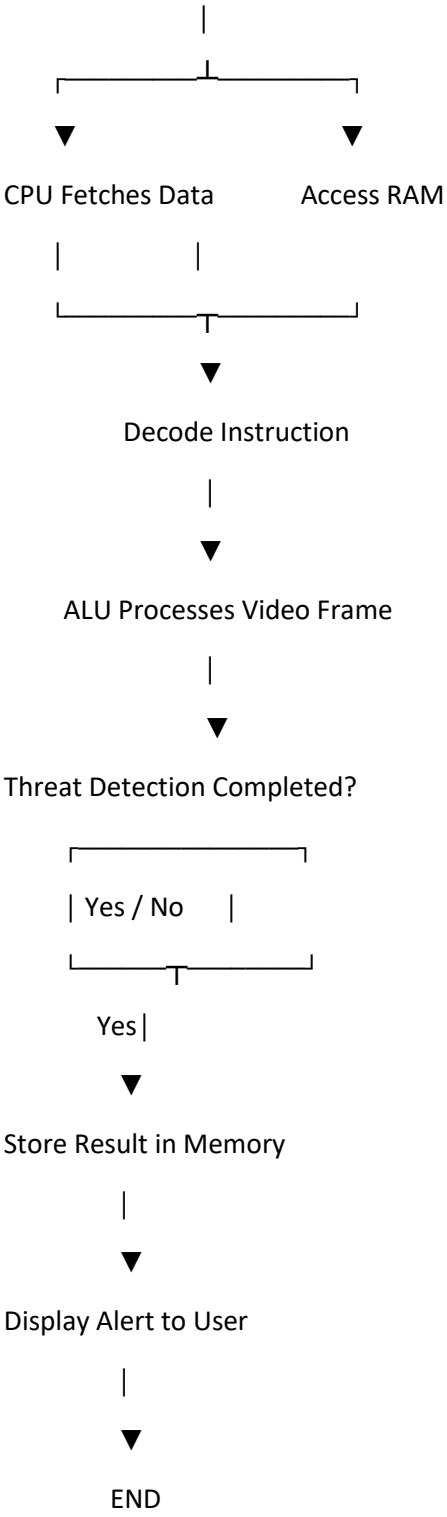
- Heavy bus traffic
- Cache misses
- Slow RAM access
- CPU waiting for memory
- Large I/O data transfer
- Bus contention
- High instruction execution time

Methods to Improve Performance

- Implement multi-bus architecture.
- Increase cache memory size.
- Use Direct Memory Access (DMA) for faster I/O transfer.
- Apply instruction pipelining.
- Use multicore processors.
- Implement interrupt-driven I/O instead of polling.
- Compress video before transmission.
- Use parallel processing for multiple camera feeds.

FLOWCHART




CPU Fetches Data
Access RAM
Decode Instruction
ALU Processes Video Frame
Threat Detection Completed?
Yes / No
Yes
Store Result in Memory
Display Alert to User
END

2. . A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

Analysis

A real-time stock trading application executes millions of instructions every second. During heavy trading, frequent memory access and branch instructions increase the **Cycles Per Instruction (CPI)**, causing delays in processing transactions.

Fetch–Decode–Execute Cycle

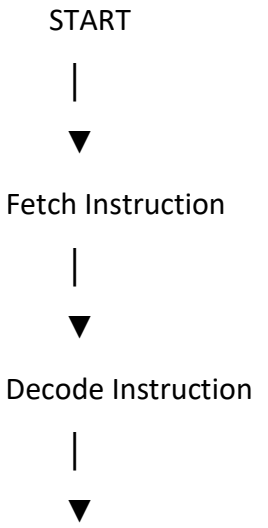
- **Fetch:** The CPU fetches the instruction from memory.
- **Decode:** The Control Unit interprets the instruction.
- **Execute:** The ALU performs the required operation.

Frequent memory access increases fetch time, while branch instructions cause pipeline stalls, resulting in slower execution.

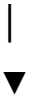
Methods to Reduce CPI

- Increase cache memory to reduce memory access.
- Use branch prediction to avoid pipeline stalls.
- Apply instruction pipelining.
- Use superscalar or multicore processors.

FLOWCHART



Execute Instruction



Store Transaction

Transaction Complete

END

C PROGRAM

```
#include <stdio.h>

int main()
{
   char *instructions[] = {
      "LOAD Stock Price",
      "COMPARE Buy/Sell",
      "STORE Transaction"
   };

   printf("=== Stock Trading Execution ===\n\n");

   for(int i=0; i<3; i++)
   {
      printf("Fetch : %s\n", instructions[i]);
      printf("Decode\n");
      printf("Execute\n\n");
```

```
    }

    printf("Transaction Completed Successfully.\n");

    return 0;
}
```

OUTPUT

```
=== Stock Trading Execution ===

Fetch : LOAD Stock Price
Decode
Execute

Fetch : COMPARE Buy/Sell
Decode
Execute

Fetch : STORE Transaction
Decode
Execute

Transaction Completed Successfully.
```

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

Analysis

An industrial temperature control unit uses an **8085 microprocessor** to read temperature values from sensors and control actuators such as cooling fans. Incorrect temperature readings may occur due to improper use of addressing modes, resulting in wrong data being accessed.

Addressing Modes in 8085

- **Immediate Addressing:** Data is directly included in the instruction.
- **Register Addressing:** Data is transferred between CPU registers.
- **Direct Addressing:** Data is accessed from a specified memory location.
- **Register Indirect Addressing:** Register pair stores the memory address.
- **Implied Addressing:** Operand is implied by the instruction.

Possible Causes of Errors

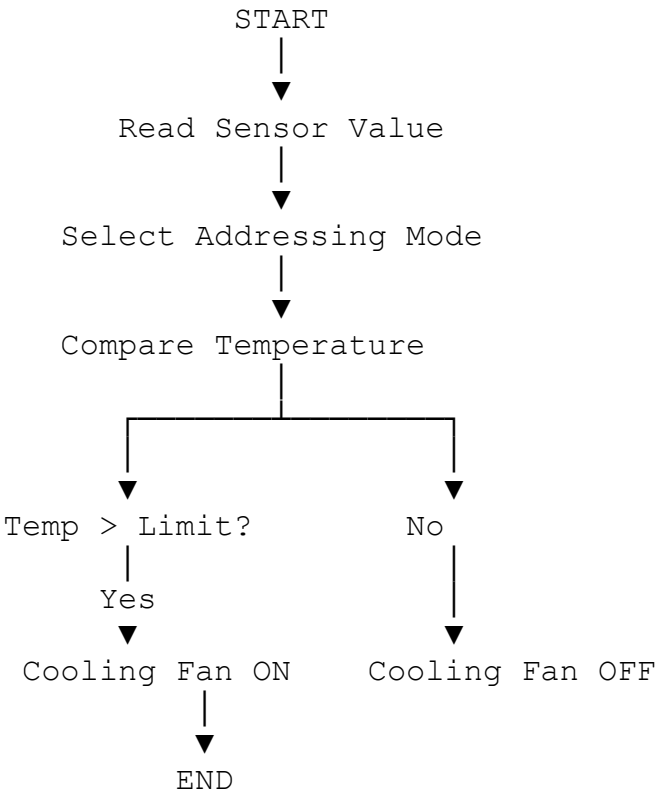
- Wrong memory address used.
- Incorrect register pair selection.
- Improper sensor data storage.
- Incorrect use of indirect addressing.

Using the 8085 Instruction Set

- **IN** – Read sensor data.
- **OUT** – Send data to actuator.
- **LDA** – Load data from memory.
- **STA** – Store data into memory.
- **CMP** – Compare temperature values.
- **JZ / JNZ** – Perform conditional branching.

Proper addressing modes and instructions ensure accurate temperature monitoring and reliable system operation.

FLOWCHART



C PROGRAM

```
#include <stdio.h>

int main()
{
    int temp;

    printf("Enter Temperature: ");
    scanf("%d",&temp);

    if(temp > 50)
        printf("Cooling Fan ON");
    else
        printf("Cooling Fan OFF");

    return 0;
}
```

OUTPUT

```
Enter Temperature: 55

Cooling Fan ON
```

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

Analysis

The **8086 microprocessor** uses **memory segmentation** to organize memory into different segments, allowing multiple programs to execute efficiently. Improper handling of these segments may lead to incorrect data access and stack overflow errors.

Memory Segments

- **Code Segment (CS):** Stores program instructions.
- **Data Segment (DS):** Stores variables and data.
- **Stack Segment (SS):** Stores function calls and local data.
- **Extra Segment (ES):** Used for additional data storage.

Problems Due to Improper Segment Handling

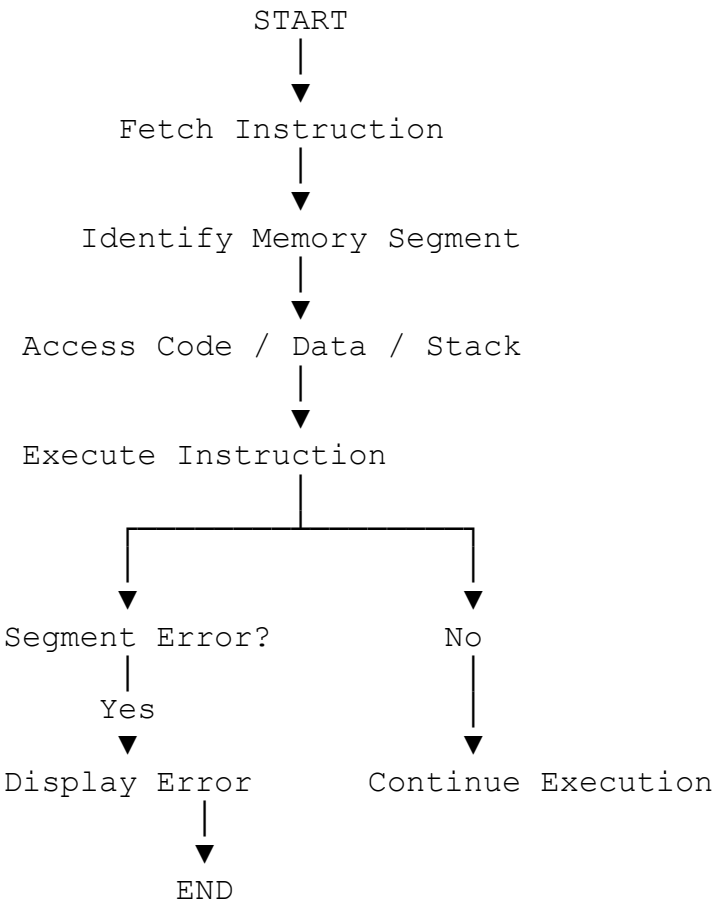
- Accessing data from the wrong segment.
- Stack overflow due to excessive push operations.
- Incorrect program execution.
- Invalid memory references.

Proper Management

- Initialize segment registers correctly.
- Use appropriate addressing modes.
- Balance PUSH and POP instructions.
- Keep code, data, and stack segments separate.

These practices ensure reliable execution of multiple programs.

Flowchart



C PROGRAM

```
#include <stdio.h>

int main()
{
    printf("8086 Memory Segmentation\n\n");

    printf("Code Segment Loaded\n");
    printf("Data Segment Accessed\n");
    printf("Stack Segment Updated\n");

    printf("\nProgram Executed Successfully");

    return 0;
}
```

OUTPUT

```
8086 Memory Segmentation

Code Segment Loaded
Data Segment Accessed
Stack Segment Updated

Program Executed Successfully
```

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

Analysis

A data communication system often represents packet information in hexadecimal format because it is easier for engineers to read, debug, and transmit compared to long binary values. In the given scenario, a software engineer incorrectly interprets hexadecimal values, which leads to wrong data processing and affects the reliability of the system.

Relationship Between Binary and Hexadecimal

Computers internally process all information in binary (0s and 1s). However, binary numbers become very long when representing memory addresses, instructions, and packet data.

Hexadecimal is a compact representation where each hexadecimal digit corresponds to 4 binary bits.

Example

Hexadecimal	Binary
0	0000
A	1010
F	1111
2A	0010 1010
FF	1111 1111

Because of this direct mapping, conversion between hexadecimal and binary is fast and does not require complex calculations.

Importance of Number System Conversion

During packet transmission, data is usually displayed in hexadecimal, while the CPU executes it in binary form.

Hexadecimal → Binary

Each hexadecimal digit is replaced with its 4-bit binary equivalent.

Example:

- $3F_{16} \rightarrow 0011\ 1111_2$

Binary → Hexadecimal

Binary bits are grouped into sets of four.

Example:

- $11010110_2 \rightarrow D6_{16}$

Accurate conversion is essential because a single incorrect bit can completely change the meaning of data.

Effect of Conversion Errors

If a hexadecimal value is interpreted incorrectly, the resulting binary pattern will also be incorrect.

Example

Correct value:

- $2A_{16} = 0010\ 1010_2$

Mistaken value:

- $2B_{16} = 0010\ 1011_2$

Although only one bit changes, the CPU interprets it as a different value. In communication systems, such an error may cause:

- Incorrect packet identification.
- Corrupted control information.
- Wrong memory addresses.
- Invalid instruction execution.
- Failure of security or authentication checks.

Impact on Instruction Execution

The CPU executes instructions through the Fetch–Decode–Execute cycle.

Fetch

The instruction is fetched from memory.

Decode

The control unit interprets the binary opcode.

Execute

The ALU performs the required operation.

If a hexadecimal instruction is converted incorrectly, the decoded opcode changes, causing the CPU to execute an unintended instruction. This may result in program crashes, incorrect calculations, or communication failures.

Why Hexadecimal is Preferred in Low-Level Programming

Hexadecimal is widely used in system programming and debugging for several reasons:

- Compact Representation: One hexadecimal digit represents four binary bits.
- Easy Readability: Memory addresses such as 7FFAH are easier to read than 0111111111111010_2.
- Direct Mapping: Simple conversion between binary and hexadecimal.
- Efficient Debugging: Engineers can quickly inspect memory dumps and registers.
- Instruction Representation: Machine code instructions are commonly shown in hexadecimal.

For example, the 8086 instruction B8 2A 00 is easier to analyze than its binary equivalent.

Role of Efficient CPU Design

Modern processors include architectural features that help detect and minimize such errors.

Cache Memory

Reduces memory access time and speeds up instruction fetching.

Error Detection Mechanisms

Parity bits and ECC memory detect transmission and storage errors.

Pipelining

Allows multiple instructions to be processed simultaneously.

Branch Prediction

Reduces delays caused by conditional instructions.

Hardware Validation

Checks instruction formats before execution.

These mechanisms improve reliability and performance in real-time communication systems.

Benchmarking and Error Detection

Benchmarking tools monitor CPU performance and identify abnormal behavior caused by incorrect data interpretation.

Metrics include:

- CPI (Cycles Per Instruction)
- Cache miss rate
- Instruction throughput
- Packet processing latency

If a conversion error causes repeated instruction failures, benchmarking reveals increased execution time and reduced throughput, allowing engineers to locate the problem quickly.

Conclusion

Hexadecimal representation plays a critical role in data communication, debugging, and low-level system design. Since computers execute instructions in binary form, accurate conversion between hexadecimal and binary is essential. Errors in conversion can alter instruction execution, corrupt packet processing, and reduce system reliability. Efficient CPU

architectures, error detection mechanisms, and benchmarking techniques help identify such problems early and ensure accurate and high-performance operation in real-time systems.

C PROGRAM

```
#include <stdio.h>

int main()
{
    int hex = 0x2A;

    printf("Hexadecimal : %X\n", hex);
    printf("Decimal     : %d\n", hex);

    return 0;
}
```

OUTPUT

```
Hexadecimal : 2A
Decimal     : 42
```